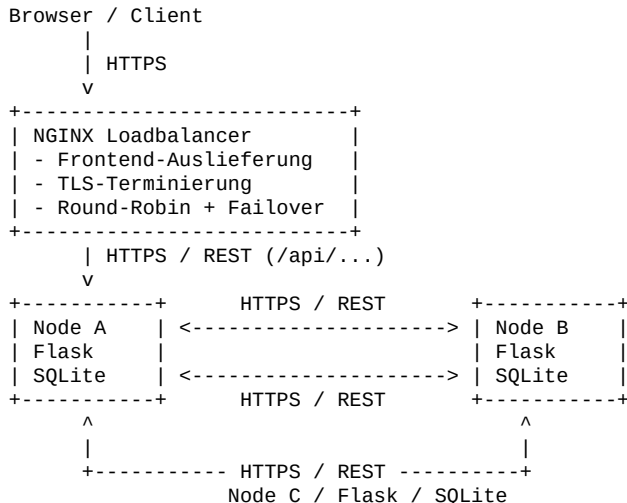


Architektur-Kurzbeschreibung - Verteiltes Commandcenter

Ziel

Das System ist ein verteilter Statusserver für textbasierte Statusmeldungen mit Geokoordinaten. Benutzer greifen über eine zentrale HTTPS-URL auf das Frontend zu. Im Backend laufen drei gleichwertige Statusnodes, die Statusobjekte lokal speichern und untereinander replizieren.

Blockdiagramm



Komponenten

Komponente	Aufgabe
Frontend	Weboberfläche mit Formular, globalem Feed und Leaflet-Karte
NGINX Loadbalancer	Single Point of Access, HTTPS, statisches Frontend, API-Failover
StatusNode A/B/C	CRUD-API, lokale SQLite-Persistenz, Replikation, Bootstrap
SQLite pro Node	Persistenter lokaler Speicher ohne gemeinsame Datenbank

Kommunikation und Sicherheit

Der Client spricht ausschließlich den Loadbalancer über HTTPS an. NGINX verteilt `/api/...`-Requests per HTTPS auf die Statusnodes. Die Nodes replizieren Schreiboperationen per `POST /replicate` und holen beim Neustart Snapshots per `GET /internal/snapshot`, ebenfalls über HTTPS. Es wird ein selbstsigniertes Dev-Zertifikat verwendet; die interne Zertifikatsprüfung ist im Compose-Setup deaktiviert, die Verbindung bleibt aber verschlüsselt.

Replikation und Konsistenz

Ein Statusobjekt wird über `username` identifiziert und enthält `statustext`, `uhrzeit`, `latitude`, `longitude`, `deleted` und `originNode`. Schreibt ein Client auf eine Node, speichert diese lokal und pusht das Update an ihre Peers. Konflikte werden deterministisch mit Last-Writer-Wins gelöst: neuere `uhrzeit` gewinnt, bei gleicher Zeit entscheidet `originNode`. Deletes werden als Tombstones repliziert, damit alte Updates gelöschte Einträge nicht wiederherstellen.

Fehlertoleranz

Fällt eine Statusnode aus, leitet NGINX Requests automatisch auf die verbleibenden Nodes weiter. Fehlgeschlagene Peer-Replikationen werden in einer Retry-Queue gespeichert und später nachgeliefert. Startet eine Node neu, befindet sie sich in einer Grace Period, zieht Peer-Snapshots, merged per Last-Writer-Wins und beantwortet erst danach Client-Anfragen.

Start und Demo

```
docker compose up --build
```

Die Anwendung ist danach unter `https://localhost:8443` erreichbar. Bei lokal gesetzter `.env` kann der Port abweichen. Das selbstsignierte Zertifikat muss im Browser akzeptiert werden.